

Cyfrowy Model Logiczny Systemu TipJar

Wprowadzenie i Legenda

Niniejszy dokument stanowi kompletną, ultraszczegółową mapę logiczną systemu TipJar, przedstawioną w formacie JSON. Został on stworzony jako cyfrowa replikacja całego systemu, odwzorowująca przepływy informacji, komponenty, działania użytkowników i logikę biznesową, dokładnie tak, jak w technicznych diagramach przepływu (BPMN, C4 model). Celem tego dokumentu jest dostarczenie jednoznacznego i gotowego do implementacji planu dla zespołu deweloperskiego, który będzie wdrażał platformę w oparciu o technologie takie jak React, Nest.js, PostgreSQL oraz usługi chmurowe AWS.

Każdy element systemu jest reprezentowany jako obiekt JSON o zdefiniowanej strukturze, co pozwala na maszynowe przetwarzanie oraz jednoznaczną interpretację poszczególnych kroków i ich wzajemnych powiązań.

Legenda Schematu JSON

Poniżej znajduje się szczegółowe wyjaśnienie kluczy używanych w obiektach JSON do opisu każdego elementu systemu:

- **id**: Unikalny, czytelny dla człowieka identyfikator węzła w systemie. Służy do tworzenia powiązań między elementami.
- **title**: Zwięzła, biznesowa nazwa elementu, opisująca jego główną funkcję.
- **type**: Kategoria elementu, która klasyfikuje jego naturę w architekturze systemu. Dostępne typy to: **user_action** (działanie inicjowane przez użytkownika w interfejsie), **ui_component** (element interfejsu), **backend_logic** (operacja logiki biznesowej na serwerze), **api_call** (wywołanie zewnętrznego API), **condition** (węzeł decyzyjny, rozgałęziający przepływ), **data_store** (operacja na bazie danych), **system_event** (zdarzenie systemowe, np. wywołanie asynchroniczne, webhook).
- **description**: Dokładne, techniczne wyjaśnienie przeznaczenia elementu, jego roli w procesie oraz kluczowych szczegółów implementacyjnych. Ten klucz zawiera najwięcej kontekstu technicznego.
- **depends_on**: Tablica identyfikatorów (**id**) elementów, które muszą zostać zakończone przed wykonaniem tego kroku. Definiuje zależności i kolejność w przepływie.
- **outputs**: Tablica identyfikatorów (**id**) elementów, które są uruchamiane po pomyślnym zakończeniu tego kroku. Definiuje dalszy ciąg przepływu.
- **related_data**: Kluczowe dane (zmienne, obiekty), które są przetwarzane, generowane lub przekazywane przez ten element.
- **visibility**: Określa poziom dostępu do danego zasobu lub akcji. Wartości to: **public** (dostępne dla wszystkich), **internal** (wymaga uwierzytelnienia, np. dla zalogowanego twórcy lub fana), **admin_only** (dostępne tylko dla administratorów platformy).

- **error_handling**: Opis zachowania systemu w przypadku wystąpienia błędu na tym etapie, w tym mechanizmy ponawiania, logowania i powiadamiania.

Metadane Systemu i Główne Ścieżki Użytkownika

Poniższy blok metadanych dostarcza ogólnego obrazu granic systemu, jego głównych punktów wejścia i wyjścia oraz sugeruje optymalną strategię wizualizacji całej mapy logicznej.

JSON

```
{
  "entry_points": [
    "creator_visits_landing_page",
    "fan_visits_creator_profile_page",
    "admin_navigates_to_login_page"
  ],
  "exit_points": [
    "creator_receives_payout_confirmation_email",
    "fan_views_tip_success_animation",
    "admin_saves_platform_configuration_and_logs_out",
    "user_session_expires_or_logs_out"
  ],
  "diagram_suggestion": {
    "type": "swimlanes",
    "grouping_strategy": "wg ról użytkowników",
    "highlight_paths":
  }
}
```

Architektura systemu TipJar jest fundamentalnie zorientowana na role użytkowników: Twórcy, Fana i Administratora. Dokumentacja funkcjonalna konsekwentnie rozdziela funkcje według tych ról, co naturalnie prowadzi do wniosku, że najklarowniejszą wizualizacją logiki systemu jest diagram typu "swimlanes" (tory pływackie), gdzie każdy tor odpowiada jednej roli (w tym rolom systemowym jak Backend, Baza Danych czy API Zewnętrzne). Taka struktura pozwala na czytelne śledzenie interakcji i przekazywania odpowiedzialności między różnymi częściami systemu. Klucz

visibility w każdym obiekcie JSON bezpośrednio odzwierciedla te tory, definiując zasady kontroli dostępu (RBAC) dla każdego zasobu i akcji w systemie.

Kompletna Mapa Logiczna Systemu (JSON)

Poniżej znajduje się główny artefakt tego dokumentu: kompletna mapa logiczna systemu TipJar w formacie JSON. Elementy są uporządkowane w sposób odzwierciedlający główne cykle życia i interakcje użytkowników.

JSON

```
{
  "system_name": "TipJar Micropayment Platform",
  "version": "1.0",
  "system_map":
},
{
  "id": "group_fan_lifecycle",
  "title": "Cykl życia Fana",
  "type": "group",
  "description": "Grupuje wszystkie procesy związane z interakcją Fana z platformą, głównie proces przekazywania napiwków.",
  "children": [
    "group_fan_tipping_process"
  ]
}
```

```

    },
    {
      "id": "group_admin_processes",
      "title": "Procesy Administracyjne",
      "type": "group",
      "description": "Grupuje procesy związane z zarządzaniem platformą przez administratorów.",
      "children": [
        "admin_navigates_to_login_page",
        "admin_submits_credentials",
        "backend_validates_admin_credentials",
        "admin_session_created",
        "admin_views_dashboard",
        "admin_configures_platform_fees",
        "backend_saves_fee_configuration",
        "db_update_platform_config",
        "admin_moderates_content",
        "backend_flags_user_content",
        "db_update_user_status_frozen",
        "admin_saves_platform_configuration_and_logs_out"
      ]
    },
    {
      "id": "group_background_systems",
      "title": "Systemy Tła i Zdarzenia",
      "type": "group",
      "description": "Grupuje procesy asynchroniczne, powiadomienia i monitoring.",
      "children": [
        "event_tip_confirmed",
        "notification_service_sends_email",
        "gamification_service_evaluates_activity",
        "analytics_service_logs_event",
        "monitoring_service_checks_health",
        "sentry_captures_unhandled_exception"
      ]
    },
    {
      "id": "group_creator_onboarding",
      "title": "Rejestracja i Logowanie Twórcy",
      "type": "group",
      "description": "Proces zakładania konta przez Twórcę za pomocą metod OAuth2 (Google) lub Sign-In with Ethereum (SIWE). Obejmuje asynchroniczne tworzenie portfela w Circle.",
      "children": [
        "creator_visits_landing_page",
        "creator_chooses_login_method",
        "path_oauth_google",
        "path_siwe"
      ]
    },
    {
      "id": "creator_visits_landing_page",
      "title": "Twórca odwiedza stronę główną",
      "type": "user_action",
      "description": "Początkowy punkt wejścia dla nowego lub powracającego twórcy. Strona prezentuje kluczowe informacje o platformie i przyciski CTA do rejestracji/logowania.",
      "depends_on": [],
      "outputs": ["creator_chooses_login_method"],
      "related_data": ["ip_address", "user_agent"],
      "visibility": "public",
      "error_handling": "Strona nie ładuje się - błąd serwera lub sieci. Użytkownik widzi standardową stronę błędu przeglądarki."
    },
    {
      "id": "creator_chooses_login_method",
      "title": "Twórca wybiera metodę logowania",
      "type": "user_action",
      "description": "Użytkownik klika przycisk 'Zaloguj przez Google' lub 'Zaloguj portfelem Web3', co inicjuje odpowiednią ścieżkę uwierzytelniania.",
      "depends_on": ["creator_visits_landing_page"],
      "outputs": ["creator_clicks_google_login", "creator_clicks_siwe_login"],

```

```

    "related_data": ["login_method_choice"],
    "visibility": "public",
    "error_handling": "Brak."
  },
  {
    "id": "path_oauth_google",
    "title": "Ścieżka logowania przez Google (OAuth2)",
    "type": "group",
    "children": [
      "creator_clicks_google_login",
      "backend_redirects_to_google_oauth",
      "user_authORIZES_in_google",
      "backend_receives_google_callback",
      "auth_service_validates_oauth_user",
      "db_check_user_by_provider_id",
      "condition_is_new_user",
      "handle_existing_user_login",
      "handle_new_user_registration"
    ]
  },
  {
    "id": "creator_clicks_google_login",
    "title": "Użytkownik klika 'Zaloguj przez Google'",
    "type": "user_action",
    "description": "Kliknięcie przycisku powoduje przekierowanie użytkownika na endpoint `/auth/google` w backendzie, który rozpoczyna proces OAuth2.",
    "depends_on": ["creator_chooses_login_method"],
    "outputs": ["backend_redirects_to_google_oauth"],
    "related_data": [],
    "visibility": "public",
    "error_handling": "Brak."
  },
  {
    "id": "backend_redirects_to_google_oauth",
    "title": "Backend przekierowuje do Google OAuth",
    "type": "backend_logic",
    "description": "Endpoint `/auth/google` w NestJS, chroniony przez `AuthGuard('google')` z biblioteki Passport.js, automatycznie generuje URL autoryzacyjny Google i przekierowuje tam przeglądarkę użytkownika. [1]",
    "depends_on": ["creator_clicks_google_login"],
    "outputs": ["user_authORIZES_in_google"],
    "related_data": ["google_oauth_url", "session_state"],
    "visibility": "internal",
    "error_handling": "Błąd konfiguracji kluczy API Google. Serwer zwraca błąd 500, błąd jest logowany w Sentry."
  },
  {
    "id": "user_authORIZES_in_google",
    "title": "Użytkownik autoryzuje aplikację w Google",
    "type": "user_action",
    "description": "Użytkownik loguje się na swoje konto Google i wyraża zgodę na udostępnienie TipJar podstawowych danych profilowych (email, nazwa, avatar).",
    "depends_on": ["backend_redirects_to_google_oauth"],
    "outputs": ["backend_receives_google_callback"],
    "related_data": ["authorization_code"],
    "visibility": "public",
    "error_handling": "Użytkownik odmawia autoryzacji. Google przekierowuje z powrotem do aplikacji z parametrem błędu. Frontend wyświetla komunikat o anulowaniu logowania."
  },
  {
    "id": "backend_receives_google_callback",
    "title": "Backend odbiera callback od Google",
    "type": "system_event",
    "description": "Google przekierowuje użytkownika na endpoint `/auth/google/callback` w backendzie, przekazując kod autoryzacyjny. Guard Passport.js przechwytuje to wywołanie i wymienia kod na token dostępu oraz dane profilu użytkownika. [1]",
    "depends_on": ["user_authORIZES_in_google"],
    "outputs": ["auth_service_validates_oauth_user"],
    "related_data": ["google_profile_data", "access_token"],

```

```

        "visibility": "internal",
        "error_handling": "Nieprawidłowy kod autoryzacyjny. Google zwraca błąd. Backend loguje błąd i przekierowuje użytkownika na stronę logowania z komunikatem o błędzie."
    },
    {
        "id": "auth_service_validates_oauth_user",
        "title": "Serwis Auth weryfikuje użytkownika OAuth",
        "type": "backend_logic",
        "description": "Metoda `validate` w `GoogleStrategy` wywołuje `AuthService.validateOAuthUser`, przekazując dane z profilu Google (`googleId`, `email`, `displayName`). Serwis sprawdza, czy użytkownik o danym `googleId` lub `email` istnieje w bazie. [1]",
        "depends_on": ["backend_receives_google_callback"],
        "outputs": ["db_check_user_by_provider_id"],
        "related_data": ["googleId", "email", "displayName", "avatarUrl"],
        "visibility": "internal",
        "error_handling": "Profil Google nie zawiera wymaganych danych (np. email). Proces jest przerywany, zwracany jest błąd autoryzacji."
    },
    {
        "id": "db_check_user_by_provider_id",
        "title": "Sprawdzenie istnienia użytkownika w bazie danych",
        "type": "data_store",
        "description": "Zapytanie do bazy danych PostgreSQL (tabela `User`) w celu znalezienia rekordu pasującego do `googleId` lub `email`. `SELECT * FROM \"User\" WHERE \"googleId\" = $1 OR \"email\" = $2`. ",
        "depends_on": ["auth_service_validates_oauth_user"],
        "outputs": ["condition_is_new_user"],
        "related_data": ["user_record_or_null"],
        "visibility": "internal",
        "error_handling": "Błąd połączenia z bazą danych. Serwer zwraca 503, błąd logowany w Sentry."
    },
    {
        "id": "condition_is_new_user",
        "title": "Warunek: Czy to nowy użytkownik?",
        "type": "condition",
        "description": "Logika warunkowa sprawdzająca, czy zapytanie do bazy danych zwróciło istniejący rekord użytkownika. Rozdziela przepływ na ścieżkę rejestracji i logowania.",
        "depends_on": ["db_check_user_by_provider_id"],
        "outputs": ["handle_new_user_registration", "handle_existing_user_login"],
        "related_data": ["is_new_user_boolean"],
        "visibility": "internal",
        "error_handling": "Brak."
    },
    {
        "id": "handle_new_user_registration",
        "title": "Obsługa rejestracji nowego użytkownika",
        "type": "group",
        "children": [
            "db_create_new_user_record",
            "async_trigger_wallet_provisioning",
            "auth_service_generates_jwt_for_user"
        ]
    },
    {
        "id": "db_create_new_user_record",
        "title": "Utworzenie nowego rekordu użytkownika w bazie",
        "type": "data_store",
        "description": "Tworzy nowy wpis w tabeli `User` z danymi z profilu Google. Generuje unikalny `username`. `INSERT INTO \"User\" (...) VALUES (...)`. [1]",
        "depends_on": ["condition_is_new_user"],
        "outputs": ["async_trigger_wallet_provisioning", "auth_service_generates_jwt_for_user"],
        "related_data": ["user_id", "email", "googleId", "displayName"],
        "visibility": "internal",
        "error_handling": "Błąd zapisu do bazy (np. naruszenie unikalności `email` lub `username`). Transakcja jest wycofywana, użytkownik otrzymuje błąd."
    },
    {
        "id": "async_trigger_wallet_provisioning",
        "title": "Asynchroniczne zlecenie utworzenia portfela",
        "type": "system_event",
    }

```

```

        "description": "Po pomyślnym utworzeniu rekordu użytkownika, system emituje zdarzenie `user.created` lub
umieszcza zadanie w kolejce (np. BullMQ + Redis). To zleca utworzenie portfela Circle w tle, nie blokując
procesu logowania. [1]",
        "depends_on": ["db_create_new_user_record"],
        "outputs": ["circle_service_provisions_user_wallet"],
        "related_data": ["user_id", "email"],
        "visibility": "internal",
        "error_handling": "Błąd połączenia z Redisem. Zadanie nie trafia do kolejki. System loguje krytyczny
błąd, może być wymagana interwencja manualna w celu utworzenia portfela."
    },
    {
        "id": "handle_existing_user_login",
        "title": "Obsługa logowania istniejącego użytkownika",
        "type": "group",
        "children": [
            "db_update_user_last_login",
            "auth_service_generates_jwt_for_user"
        ]
    },
    {
        "id": "db_update_user_last_login",
        "title": "Aktualizacja daty ostatniego logowania",
        "type": "data_store",
        "description": "Aktualizuje pole `lastLoginAt` w rekordzie istniejącego użytkownika. `UPDATE `User` SET
`lastLoginAt` = NOW() WHERE id = $1`.",
        "depends_on": ["condition_is_new_user"],
        "outputs": ["auth_service_generates_jwt_for_user"],
        "related_data": ["user_id"],
        "visibility": "internal",
        "error_handling": "Błąd zapisu do bazy. Logowanie jest kontynuowane, ale data nie jest aktualizowana.
Błąd logowany w Sentry."
    },
    {
        "id": "auth_service_generates_jwt_for_user",
        "title": "Serwis Auth generuje token JWT",
        "type": "backend_logic",
        "description": "Niezależnie od ścieżki (rejestracja/logowanie), `AuthService` używa `@nestjs/jwt` do
wygenerowania podpisanego tokenu JWT zawierającego `userId` i `role`. Token ten będzie używany do
uwierzytelniania kolejnych żądań API. [1]",
        "depends_on": ["db_create_new_user_record", "db_update_user_last_login"],
        "outputs": ["backend_sets_jwt_cookie_and_redirects"],
        "related_data": ["jwt_token", "user_id"],
        "visibility": "internal",
        "error_handling": "Błąd podczas generowania tokenu (np. brak sekretu JWT w konfiguracji). Serwer zwraca
błąd 500."
    },
    {
        "id": "backend_sets_jwt_cookie_and_redirects",
        "title": "Backend ustawia JWT w cookie i przekierowuje",
        "type": "backend_logic",
        "description": "W odpowiedzi na callback, backend ustawia wygenerowany token JWT w bezpiecznym ciasteczku
`HttpOnly` i przekierowuje przeglądarkę użytkownika na stronę panelu twórcy (`/dashboard`). [1]",
        "depends_on": ["auth_service_generates_jwt_for_user"],
        "outputs": ["creator_lands_on_dashboard"],
        "related_data": ["jwt_token_cookie"],
        "visibility": "internal",
        "error_handling": "Brak."
    },
    {
        "id": "path_siwe",
        "title": "Ścieżka logowania przez Sign-In with Ethereum",
        "type": "group",
        "children": [
            "creator_clicks_siwe_login",
            "frontend_requests_nonce",
            "backend_generates_nonce",
            "frontend_creates_siwe_message",
            "user_signs_siwe_message_in_wallet",
            "frontend_sends_signature_for_verification",

```

```

        "backend_verifies_siwe_signature",
        "db_check_user_by_wallet_address",
        "condition_is_new_user_siwe",
        "handle_new_user_registration_siwe",
        "handle_existing_user_login_siwe"
    ]
},
{
    "id": "creator_clicks_siwe_login",
    "title": "Użytkownik klika 'Zaloguj portfelem Web3'",
    "type": "user_action",
    "description": "Kliknięcie przycisku inicjuje logikę SIWE po stronie frontentu. Aplikacja sprawdza
dostępność dostawcy Ethereum (np. `window.ethereum`). [1]",
    "depends_on": ["creator_chooses_login_method"],
    "outputs": ["frontend_requests_nonce"],
    "related_data":,
    "visibility": "public",
    "error_handling": "Brak zainstalowanego portfela. Frontend wyświetla komunikat 'Zainstaluj MetaMask, aby
kontynuować'."
},
{
    "id": "frontend_requests_nonce",
    "title": "Frontend żąda nonce od backendu",
    "type": "api_call",
    "description": "Frontend wysyła żądanie GET na endpoint `/auth/siwe/nonce`, aby otrzymać unikalny,
jednorazowy ciąg znaków do podpisania, co zapobiega atakom typu replay. [1]",
    "depends_on": ["creator_clicks_siwe_login"],
    "outputs": ["backend_generates_nonce"],
    "related_data":,
    "visibility": "public",
    "error_handling": "Błąd sieci lub serwera. Frontend wyświetla komunikat o błędzie."
},
{
    "id": "backend_generates_nonce",
    "title": "Backend generuje i zwraca nonce",
    "type": "backend_logic",
    "description": "Endpoint `/auth/siwe/nonce` generuje kryptograficznie bezpieczny, losowy ciąg znaków,
tymczasowo go przechowuje (np. w sesji lub Redis) i zwraca do frontentu. [1]",
    "depends_on": ["frontend_requests_nonce"],
    "outputs": ["frontend_creates_siwe_message"],
    "related_data": ["nonce"],
    "visibility": "internal",
    "error_handling": "Błąd generatora losowego. Serwer zwraca 500."
},
{
    "id": "frontend_creates_siwe_message",
    "title": "Frontend tworzy wiadomość SIWE",
    "type": "ui_component",
    "description": "Frontend, używając biblioteki `siwe`, konstruuje wiadomość zgodną ze standardem EIP-4361,
zawierającą domenę, adres użytkownika, oświadczenie, URI, wersję i otrzymany nonce. [1]",
    "depends_on": ["backend_generates_nonce"],
    "outputs": ["user_signs_siwe_message_in_wallet"],
    "related_data": ["siwe_message_object"],
    "visibility": "public",
    "error_handling": "Brak."
},
{
    "id": "user_signs_siwe_message_in_wallet",
    "title": "Użytkownik podpisuje wiadomość w portfelu",
    "type": "user_action",
    "description": "Frontend wywołuje `signer.signMessage()`, co powoduje wyświetlenie w portfelu MetaMask
okna z prośbą o podpisanie przygotowanej wiadomości. Użytkownik zatwierdza operację. [1]",
    "depends_on": ["frontend_creates_siwe_message"],
    "outputs": ["frontend_sends_signature_for_verification"],
    "related_data": ["signature"],
    "visibility": "public",
    "error_handling": "Użytkownik odrzuca podpis. Frontend anuluje proces logowania."
},
{

```

```

        "id": "frontend_sends_signature_for_verification",
        "title": "Frontend wysyła podpis do weryfikacji",
        "type": "api_call",
        "description": "Frontend wysyła żądanie POST na endpoint `/auth/siwe/verify` z oryginalną wiadomością i podpisem. [1]",
        "depends_on": ["user_signs_siwe_message_in_wallet"],
        "outputs": ["backend_verifies_siwe_signature"],
        "related_data": ["message", "signature"],
        "visibility": "public",
        "error_handling": "Błąd sieci. Frontend wyświetla komunikat o błędzie."
    },
    {
        "id": "backend_verifies_siwe_signature",
        "title": "Backend weryfikuje podpis SIWE",
        "type": "backend_logic",
        "description": "Endpoint `/auth/siwe/verify` używa biblioteki `ethers.js` do odzyskania adresu z podpisu (`verifyMessage`). Porównuje odzyskany adres z adresem w wiadomości, weryfikuje domenę, URI i sprawdza, czy nonce jest prawidłowy i nie został wcześniej użyty. [1]",
        "depends_on": ["frontend_sends_signature_for_verification"],
        "outputs": ["db_check_user_by_wallet_address"],
        "related_data": ["verified_wallet_address"],
        "visibility": "internal",
        "error_handling": "Podpis nieprawidłowy, nonce zużyty lub nieprawidłowy. Serwer zwraca błąd 401 Unauthorized."
    },
    {
        "id": "db_check_user_by_wallet_address",
        "title": "Sprawdzenie istnienia użytkownika po adresie portfela",
        "type": "data_store",
        "description": "Zapytanie do bazy danych PostgreSQL (tabela `User`) w celu znalezienia rekordu z pasującym `walletAddressSIWE`. `SELECT * FROM \"User\" WHERE \"walletAddressSIWE\" = $1`. ",
        "depends_on": ["backend_verifies_siwe_signature"],
        "outputs": ["condition_is_new_user_siwe"],
        "related_data": ["user_record_or_null"],
        "visibility": "internal",
        "error_handling": "Błąd połączenia z bazą danych. Serwer zwraca 503."
    },
    {
        "id": "condition_is_new_user_siwe",
        "title": "Warunek: Czy to nowy użytkownik (SIWE)?",
        "type": "condition",
        "description": "Logika warunkowa rozdzielająca przepływ na rejestrację i logowanie na podstawie wyniku z bazy danych.",
        "depends_on": ["db_check_user_by_wallet_address"],
        "outputs": ["handle_new_user_registration_siwe", "handle_existing_user_login_siwe"],
        "related_data": ["is_new_user_boolean"],
        "visibility": "internal",
        "error_handling": "Brak."
    },
    {
        "id": "handle_new_user_registration_siwe",
        "title": "Obsługa rejestracji nowego użytkownika (SIWE)",
        "type": "group",
        "children": [
            "db_create_new_user_record_siwe",
            "async_trigger_wallet_provisioning",
            "auth_service_generates_jwt_for_user"
        ]
    },
    {
        "id": "db_create_new_user_record_siwe",
        "title": "Utworzenie nowego rekordu użytkownika w bazie (SIWE)",
        "type": "data_store",
        "description": "Tworzy nowy wpis w tabeli `User` z polem `walletAddressSIWE` i wygenerowanym `username`. `INSERT INTO \"User\" (`walletAddressSIWE`, `username`) VALUES ($1, $2)`. ",
        "depends_on": ["condition_is_new_user_siwe"],
        "outputs": ["async_trigger_wallet_provisioning", "auth_service_generates_jwt_for_user"],
        "related_data":,
        "visibility": "internal",

```

```

    "error_handling": "Błąd zapisu do bazy. Transakcja jest wycofywana."
  },
  {
    "id": "handle_existing_user_login_siwe",
    "title": "Obsługa logowania istniejącego użytkownika (SIWE)",
    "type": "group",
    "children": [
      "db_update_user_last_login",
      "auth_service_generates_jwt_for_user"
    ]
  },
  {
    "id": "circle_service_provisions_user_wallet",
    "title": "Serwis Circle tworzy portfel użytkownika",
    "type": "backend_logic",
    "description": "Worker z kolejki zadań wywołuje `CircleService.provisionUserWallet`. Ta metoda
przygotowuje i wysyła żądanie do Circle API w celu utworzenia nowego portfela typu SCA (Sponsor Controlled
Account) na sieci Polygon. [1]",
    "depends_on": ["async_trigger_wallet_provisioning"],
    "outputs": ["api_call_circle_create_wallet"],
    "related_data": ["user_id", "idempotencyKey"],
    "visibility": "internal",
    "error_handling": "Błąd logiki przygotowania żądania. Logowany w Sentry."
  },
  {
    "id": "api_call_circle_create_wallet",
    "title": "Wywołanie API Circle w celu utworzenia portfela",
    "type": "api_call",
    "description": "Faktyczne wywołanie HTTP POST do endpointu `wallets` w Circle Programmable Wallets API.
Żądanie zawiera `walletSetId`, `idempotencyKey` i konfigurację portfela. [1]",
    "depends_on": ["circle_service_provisions_user_wallet"],
    "outputs": ["db_update_user_with_wallet_details"],
    "related_data": ["circle_wallet_response"],
    "visibility": "internal",
    "error_handling": "API Circle zwraca błąd (np. 4xx, 5xx). System ponawia żądanie z mechanizmem
exponential backoff. Po 3 nieudanych próbach zadanie trafia do Dead Letter Queue i generowany jest alert dla
zespołu technicznego."
  },
  {
    "id": "db_update_user_with_wallet_details",
    "title": "Aktualizacja rekordu użytkownika o dane portfela",
    "type": "data_store",
    "description": "Po pomyślnym utworzeniu portfela, serwis zapisuje otrzymane `circleWalletId` i
`mainWalletAddress` w rekordzie użytkownika w bazie danych. `UPDATE `User` SET `circleWalletId` = $1,
`mainWalletAddress` = $2 WHERE id = $3`. [1]",
    "depends_on": ["api_call_circle_create_wallet"],
    "outputs": ["ui_updates_wallet_status"],
    "related_data": ["user_id", "circleWalletId", "mainWalletAddress"],
    "visibility": "internal",
    "error_handling": "Błąd zapisu do bazy. Generowany jest alert, ponieważ istnieje niespójność danych
między TipJar a Circle."
  },
  {
    "id": "group_creator_dashboard_management",
    "title": "Zarządzanie Panelem Twórcy",
    "type": "group",
    "children": [
      "creator_lands_on_dashboard",
      "api_fetch_dashboard_data",
      "ui_renders_dashboard_stats",
      "ui_updates_wallet_status",
      "creator_edits_profile",
      "api_update_user_profile",
      "db_save_profile_changes",
      "creator_sets_fundraising_goal",
      "api_update_fundraising_goal",
      "db_save_fundraising_goal"
    ]
  }
},

```

```

{
  "id": "creator_lands_on_dashboard",
  "title": "Twórca łąduje w panelu administracyjnym",
  "type": "user_action",
  "description": "Po pomyślnym zalogowaniu, użytkownik jest przekierowywany do `/dashboard`. Frontend inicjuje pobieranie danych panelu.",
  "depends_on": ["backend_sets_jwt_cookie_and_redirects"],
  "outputs": ["api_fetch_dashboard_data"],
  "related_data": ["jwt_token_cookie"],
  "visibility": "internal",
  "error_handling": "Brak."
},
{
  "id": "api_fetch_dashboard_data",
  "title": "API pobiera dane do panelu",
  "type": "api_call",
  "description": "Frontend wysyła żądanie do chronionego endpointu API (np. `/api/dashboard/summary`) w celu pobrania salda, historii ostatnich napiwków i statystyk. [1]",
  "depends_on": ["creator_lands_on_dashboard"],
  "outputs": ["ui_renders_dashboard_stats"],
  "related_data": ["dashboard_data_payload"],
  "visibility": "internal",
  "error_handling": "Nieprawidłowy lub wygasły token JWT. API zwraca 401, frontend przekierowuje na stronę logowania."
},
{
  "id": "ui_renders_dashboard_stats",
  "title": "UI renderuje statystyki w panelu",
  "type": "ui_component",
  "description": "Komponenty React w panelu twórcy wyświetlają otrzymane dane: saldo USDC, listę transakcji, wykresy analityczne. [1]",
  "depends_on": ["api_fetch_dashboard_data"],
  "outputs": [],
  "related_data": ["balance", "tip_history"],
  "visibility": "internal",
  "error_handling": "Błąd renderowania komponentu. Sentry loguje błąd frontendu."
},
{
  "id": "ui_updates_wallet_status",
  "title": "UI aktualizuje status portfela",
  "type": "ui_component",
  "description": "Jeśli portfel był tworzony asynchronicznie, UI może okresowo odpytywać backend o status lub nasłuchiwać na zdarzenie WebSocket. Po otrzymaniu `circleWalletId`, UI zmienia status z 'Tworzenie portfela...' na 'Aktywny' i wyświetla adres. To kluczowy element obsługi stanu przejściowego.",
  "depends_on": ["db_update_user_with_wallet_details"],
  "outputs": [],
  "related_data": ["wallet_status"],
  "visibility": "internal",
  "error_handling": "Brak."
},
{
  "id": "group_fan_tipping_process",
  "title": "Proces Przekazywania Napiwku",
  "type": "group",
  "description": "Kompletny przepływ od odwiedzenia profilu twórcy przez fana do pomyślnego przekazania napiwku. Obejmuje trzy główne metody płatności. [1]",
  "children": [
    "fan_visits_creator_profile_page",
    "fan_fills_tip_form",
    "fan_chooses_payment_method",
    "path_tip_onchain",
    "path_tip_fiat",
    "path_tip_internal"
  ]
},
{
  "id": "fan_visits_creator_profile_page",
  "title": "Fan odwiedza publiczny profil twórcy",
  "type": "user_action",

```

```

      "description": "Fan trafia na stronę `tipjar.com/@nazwa` poprzez link lub kod QR. Strona jest renderowana po stronie serwera (SSR w Next.js) dla lepszego SEO i wydajności. [1]",
      "depends_on": [],
      "outputs": ["fan_fills_tip_form"],
      "related_data": ["creator_username"],
      "visibility": "public",
      "error_handling": "Profil o danej nazwie nie istnieje. Serwer zwraca stronę 404."
    },
    {
      "id": "fan_fills_tip_form",
      "title": "Fan wypełnia formularz napiwku",
      "type": "user_action",
      "description": "Fan wybiera kwotę napiwku (suwak lub pole tekstowe) i opcjonalnie wpisuje wiadomość oraz zaznacza opcję anonimowości. [1]",
      "depends_on": ["fan_visits_creator_profile_page"],
      "outputs": ["fan_chooses_payment_method"],
      "related_data": ["amount", "message", "is_anonymous"],
      "visibility": "public",
      "error_handling": "Walidacja frontendu - kwota musi być w dozwolonym zakresie (np. 1-100 USDC)."
    },
    {
      "id": "fan_chooses_payment_method",
      "title": "Fan wybiera metodę płatności",
      "type": "user_action",
      "description": "Fan klika jeden z przycisków płatności: 'Zapłać krypto (USDC)', 'Zapłać kartą' lub 'Użyj salda TipJar' (jeśli jest zalogowany).",
      "depends_on": ["fan_fills_tip_form"],
      "outputs": ["tip_onchain_connect_wallet", "tip_fiat_initiate_payment", "tip_internal_initiate_transfer"],
      "related_data": ["payment_method_choice"],
      "visibility": "public",
      "error_handling": "Brak."
    },
    {
      "id": "path_tip_onchain",
      "title": "Ścieżka płatności krypto (On-chain)",
      "type": "group",
      "children": [
        "tip_onchain_connect_wallet",
        "tip_onchain_initiate_in_backend",
        "tip_onchain_frontend_sends_transaction",
        "tip_onchain_backend_monitors_blockchain",
        "db_create_tip_record_pending",
        "db_update_tip_record_confirmed",
        "fan_views_tip_success_animation"
      ]
    },
    {
      "id": "tip_onchain_connect_wallet",
      "title": "Fan łączy portfel MetaMask",
      "type": "user_action",
      "description": "Frontend wykrywa `window.ethereum` i prosi o połączenie konta. Sprawdza, czy sieć jest poprawna (np. Polygon) i w razie potrzeby prosi o jej zmianę. [1]",
      "depends_on": ["fan_chooses_payment_method"],
      "outputs": ["tip_onchain_initiate_in_backend"],
      "related_data": ["fan_wallet_address"],
      "visibility": "public",
      "error_handling": "Użytkownik odmawia połączenia lub jest na nieobsługiwanej sieci. UI wyświetla odpowiedni komunikat."
    },
    {
      "id": "tip_onchain_initiate_in_backend",
      "title": "Backend inicjuje transakcję on-chain",
      "type": "api_call",
      "description": "Frontend wysyła żądanie do `/tips/initiate` z danymi napiwku. Backend pobiera z bazy adres portfela twórcy (`mainWalletAddress`) i tworzy w bazie rekord napiwku ze statusem `PENDING`. [1]",
      "depends_on": ["tip_onchain_connect_wallet"],
      "outputs": ["db_create_tip_record_pending", "tip_onchain_frontend_sends_transaction"],
      "related_data": ["creator_wallet_address", "tip_id"],
      "visibility": "public",

```

```

    "error_handling": "Twórca nie ma skonfigurowanego portfela. API zwraca błąd."
  },
  {
    "id": "db_create_tip_record_pending",
    "title": "Utworzenie rekordu napiwku w stanie 'Pending'",
    "type": "data_store",
    "description": "Zapisuje nową transakcję w tabeli `Tip` ze statusem `PENDING` lub `INITIATED`.",
    "depends_on": ["tip_onchain_initiate_in_backend"],
    "outputs": [],
    "related_data": ["tip_id", "amount", "toUserId"],
    "visibility": "internal",
    "error_handling": "Błąd zapisu do bazy."
  },
  {
    "id": "tip_onchain_frontend_sends_transaction",
    "title": "Frontend wysłała transakcję USDC",
    "type": "ui_component",
    "description": "Frontend, używając `ethers.js`, wywołuje metodę `transfer` na kontrakcie USDC, przekazując adres twórcy i kwotę. MetaMask prosi fana o zatwierdzenie transakcji. Fan pokrywa koszt gazu. [1]",
    "depends_on": ["tip_onchain_initiate_in_backend"],
    "outputs": ["tip_onchain_backend_monitors_blockchain"],
    "related_data": ["transaction_hash"],
    "visibility": "public",
    "error_handling": "Użytkownik odrzuca transakcję w MetaMask. Backend jest powiadamiany o anulowaniu, status napiwku w bazie jest zmieniany na `FAILED`."
  },
  {
    "id": "tip_onchain_backend_monitors_blockchain",
    "title": "Backend monitoruje blockchain",
    "type": "system_event",
    "description": "Backend nasłuchuje na zdarzenia transferu na kontrakcie USDC (dla adresów portfeli twórców) lub otrzymuje webhook od Circle o nowym depozycie. Czeką na potwierdzenie transakcji. [1]",
    "depends_on": ["tip_onchain_frontend_sends_transaction"],
    "outputs": ["db_update_tip_record_confirmed"],
    "related_data": ["confirmed_transaction_hash"],
    "visibility": "internal",
    "error_handling": "Transakcja nie zostaje potwierdzona w określonym czasie. Status napiwku może zostać zmieniony na `FAILED` po timeout'cie."
  },
  {
    "id": "path_tip_fiat",
    "title": "Ścieżka płatności kartą (Fiat)",
    "type": "group",
    "children": [
      "tip_fiat_initiate_payment",
      "api_call_circle_create_payment_intent",
      "fan_completes_payment_in_checkout",
      "backend_receives_payment_webhook",
      "backend_credits_creator_wallet",
      "db_update_tip_record_confirmed",
      "fan_views_tip_success_animation"
    ]
  },
  {
    "id": "tip_fiat_initiate_payment",
    "title": "Backend inicjuje płatność kartą",
    "type": "api_call",
    "description": "Frontend wysłała żądanie do backendu. Backend, poprzez `CircleService`, tworzy `PaymentIntent` w Circle Payments API. [1]",
    "depends_on": ["fan_chooses_payment_method"],
    "outputs": ["api_call_circle_create_payment_intent"],
    "related_data": ["creator_id", "amount"],
    "visibility": "public",
    "error_handling": "Błąd komunikacji z Circle API. API zwraca błąd."
  },
  {
    "id": "api_call_circle_create_payment_intent",
    "title": "Wywołanie API Circle w celu utworzenia intencji płatności",
    "type": "api_call",

```

```

        "description": "Backend wysyła żądanie do Circle, które zwraca token klienta lub link do hostowanej
strony płatności. Frontend używa tego do wyświetlenia formularza karty. [1]",
        "depends_on": ["tip_fiat_initiate_payment"],
        "outputs": ["fan_completes_payment_in_checkout"],
        "related_data": ["client_token", "checkout_url"],
        "visibility": "internal",
        "error_handling": "Circle API zwraca błąd. Proces płatności jest przerywany."
    },
    {
        "id": "fan_completes_payment_in_checkout",
        "title": "Fan finalizuje płatność w checkout",
        "type": "user_action",
        "description": "Fan wprowadza dane karty i autoryzuje płatność (np. 3D Secure) w interfejsie dostarczonym
przez Circle. [1]",
        "depends_on": ["api_call_circle_create_payment_intent"],
        "outputs": ["backend_receives_payment_webhook"],
        "related_data": ["payment_authorization"],
        "visibility": "public",
        "error_handling": "Płatność odrzucona przez bank. Circle informuje o błędzie, frontend wyświetla
komunikat."
    },
    {
        "id": "backend_receives_payment_webhook",
        "title": "Backend odbiera webhook o płatności",
        "type": "system_event",
        "description": "Circle, po pomyślnej autoryzacji, wysyła webhook (np. `payment.confirmed`) na dedykowany
endpoint w backendzie TipJar. [1]",
        "depends_on": ["fan_completes_payment_in_checkout"],
        "outputs": ["backend_credits_creator_wallet"],
        "related_data": ["payment_id", "amount", "status"],
        "visibility": "internal",
        "error_handling": "Webhook nie dociera lub jest nieprawidłowy. System może mieć mechanizm odpytywania o
status płatności jako fallback."
    },
    {
        "id": "backend_credits_creator_wallet",
        "title": "Backend zasila portfel twórcy",
        "type": "backend_logic",
        "description": "Na podstawie webhooka, backend identyfikuje transakcję. Circle automatycznie zamienia
fiat na USDC i zasila portfel twórcy. Backend TipJar weryfikuje ten fakt i finalizuje operację w swoim
systemie. [1]",
        "depends_on": ["backend_receives_payment_webhook"],
        "outputs": ["db_update_tip_record_confirmed"],
        "related_data": ["creator_wallet_id", "amount_usdc"],
        "visibility": "internal",
        "error_handling": "Błąd zasilenia portfela po stronie Circle. Generowany jest alert."
    },
    {
        "id": "path_tip_internal",
        "title": "Ścieżka płatności wewnętrznej (z salda TipJar)",
        "type": "group",
        "children": [
            "tip_internal_initiate_transfer",
            "backend_verifies_fan_balance",
            "api_call_circle_internal_transfer",
            "db_update_tip_record_confirmed",
            "db_update_user_balances",
            "fan_views_tip_success_animation"
        ]
    },
    {
        "id": "tip_internal_initiate_transfer",
        "title": "Fan inicjuje transfer wewnętrzny",
        "type": "api_call",
        "description": "Zalogowany fan klika 'Wyślij napiwek'. Frontend wysyła żądanie na chroniony endpoint
`/tips/transfer`. [1]",
        "depends_on": ["fan_chooses_payment_method"],
        "outputs": ["backend_verifies_fan_balance"],
        "related_data": ["jwt_token", "creator_id", "amount"],
    }

```

```

        "visibility": "internal",
        "error_handling": "Brak tokenu JWT. API zwraca 401."
    },
    {
        "id": "backend_verifies_fan_balance",
        "title": "Backend weryfikuje saldo fana",
        "type": "backend_logic",
        "description": "Backend, na podstawie `userId` z JWT, sprawdza saldo fana w bazie danych lub poprzez API Circle. [1]",
        "depends_on": ["tip_internal_initiate_transfer"],
        "outputs": ["api_call_circle_internal_transfer"],
        "related_data": ["fan_id", "fan_balance"],
        "visibility": "internal",
        "error_handling": "Niewystarczające środki. API zwraca błąd 400, frontend wyświetla komunikat."
    },
    {
        "id": "api_call_circle_internal_transfer",
        "title": "Wywołanie API Circle w celu transferu wewnętrznego",
        "type": "api_call",
        "description": "Backend wywołuje endpoint `Transfers` w API Circle, podając `circleWalletId` fana jako źródło i `circleWalletId` twórcy jako cel. Jest to operacja off-chain, natychmiastowa. [1]",
        "depends_on": ["backend_verifies_fan_balance"],
        "outputs": ["db_update_tip_record_confirmed", "db_update_user_balances"],
        "related_data": ["from_wallet_id", "to_wallet_id", "amount"],
        "visibility": "internal",
        "error_handling": "API Circle zwraca błąd. Transakcja jest wycofywana, API zwraca 502."
    },
    {
        "id": "db_update_user_balances",
        "title": "Aktualizacja sald użytkowników w bazie",
        "type": "data_store",
        "description": "Po pomyślnym transferze wewnętrznym, backend aktualizuje lokalnie przechowywane salda fana i twórcy w tabeli `User`.",
        "depends_on": ["api_call_circle_internal_transfer"],
        "outputs": ,
        "related_data": ["fan_id", "creator_id", "new_balances"],
        "visibility": "internal",
        "error_handling": "Błąd zapisu do bazy. Generowany jest alert o niespójności sald."
    },
    {
        "id": "db_update_tip_record_confirmed",
        "title": "Aktualizacja rekordu napiwku na 'Confirmed'",
        "type": "data_store",
        "description": "Po pomyślnym zakończeniu dowolnej ścieżki płatności, status rekordu w tabeli `Tip` jest zmieniany na `CONFIRMED`. Zapisywany jest `txHash` (dla on-chain) lub `paymentId` (dla fiat).",
        "depends_on": ["tip_onchain_backend_monitors_blockchain", "backend_credits_creator_wallet", "api_call_circle_internal_transfer"],
        "outputs": ["event_tip_confirmed", "fan_views_tip_success_animation"],
        "related_data": ["tip_id", "status_confirmed"],
        "visibility": "internal",
        "error_handling": "Błąd zapisu do bazy."
    },
    {
        "id": "fan_views_tip_success_animation",
        "title": "Fan widzi animację potwierdzającą sukces",
        "type": "ui_component",
        "description": "Frontend, po otrzymaniu potwierdzenia z backendu, wyświetla wizualne potwierdzenie, np. animację 'Dziękujemy za wsparcie!'. [1]",
        "depends_on": ["db_update_tip_record_confirmed"],
        "outputs": ,
        "related_data": ,
        "visibility": "public",
        "error_handling": "Brak."
    },
    {
        "id": "group_creator_payout",
        "title": "Proces Wypłaty Środków przez Twórcę",
        "type": "group",
        "description": "Przepływ inicjowany przez twórcę w celu wypłaty zgromadzonych środków na zewnętrzne konto

```

```

krypto lub bankowe. [1]",
  "children": [
    "creator_navigates_to_payouts",
    "creator_submits_payout_request",
    "backend_validates_payout_request",
    "condition_payout_method",
    "path_payout_crypto",
    "path_payout_fiat"
  ]
},
{
  "id": "creator_navigates_to_payouts",
  "title": "Twórca przechodzi do sekcji Wypłaty",
  "type": "user_action",
  "description": "W panelu administracyjnym twórca wybiera opcję wypłaty środków.",
  "depends_on": ["creator_lands_on_dashboard"],
  "outputs": ["creator_submits_payout_request"],
  "related_data":,
  "visibility": "internal",
  "error_handling": "Brak."
},
{
  "id": "creator_submits_payout_request",
  "title": "Twórca zleca wypłatę",
  "type": "user_action",
  "description": "Twórca wprowadza kwotę oraz dane docelowe (adres portfela krypto lub dane konta bankowego) i klika 'Wypłać'. [1]",
  "depends_on": ["creator_navigates_to_payouts"],
  "outputs": ["backend_validates_payout_request"],
  "related_data": ["amount", "destination_address", "payout_method"],
  "visibility": "internal",
  "error_handling": "Walidacja frontendu (nieprawidłowy format adresu, kwota poniżej minimum).",
},
{
  "id": "backend_validates_payout_request",
  "title": "Backend waliduje żądanie wypłaty",
  "type": "backend_logic",
  "description": "Backend odbiera żądanie `/payout`, weryfikuje JWT twórcy, sprawdza jego saldo i waliduje poprawność danych docelowych. [1]",
  "depends_on": ["creator_submits_payout_request"],
  "outputs": ["db_create_withdrawal_record_pending", "condition_payout_method"],
  "related_data": ["user_id", "amount", "balance"],
  "visibility": "internal",
  "error_handling": "Niewystarczające saldo lub nieprawidłowe dane. API zwraca błąd 400."
},
{
  "id": "db_create_withdrawal_record_pending",
  "title": "Utworzenie rekordu wypłaty w stanie 'Pending'",
  "type": "data_store",
  "description": "System tworzy nowy wpis w tabeli `Withdrawal` ze statusem `PENDING` i zamraża odpowiednią kwotę na saldzie twórcy.",
  "depends_on": ["backend_validates_payout_request"],
  "outputs":,
  "related_data": ["withdrawal_id"],
  "visibility": "internal",
  "error_handling": "Błąd zapisu do bazy."
},
{
  "id": "condition_payout_method",
  "title": "Warunek: Jaka metoda wypłaty?",
  "type": "condition",
  "description": "Rozgałęzienie logiki w zależności od wybranej przez twórcę metody wypłaty: krypto czy fiat.",
  "depends_on": ["backend_validates_payout_request"],
  "outputs": ["payout_crypto_initiate_transfer", "payout_fiat_check_kyc"],
  "related_data": ["payout_method"],
  "visibility": "internal",
  "error_handling": "Brak."
},

```

```

{
  "id": "path_payout_crypto",
  "title": "Ścieżka wypłaty krypto (On-chain)",
  "type": "group",
  "children": [
    "payout_crypto_initiate_transfer",
    "api_call_circle_payout_onchain",
    "backend_awaits_payout_webhook",
    "db_update_withdrawal_record_confirmed",
    "creator_receives_payout_confirmation_email"
  ]
},
{
  "id": "payout_crypto_initiate_transfer",
  "title": "Backend inicjuje transfer on-chain",
  "type": "backend_logic",
  "description": "Backend, poprzez `CircleService`, przygotowuje żądanie transferu z portfela twórcy na zewnętrzny adres. [1]",
  "depends_on": ["condition_payout_method"],
  "outputs": ["api_call_circle_payout_onchain"],
  "related_data": ["from_wallet_id", "to_external_address", "amount"],
  "visibility": "internal",
  "error_handling": "Błąd logiki."
},
{
  "id": "api_call_circle_payout_onchain",
  "title": "Wywołanie API Circle w celu wypłaty on-chain",
  "type": "api_call",
  "description": "Wywołanie API Circle (np. Payouts lub Transfers z flagą `external`). Circle inicjuje transakcję on-chain, używając `Gas Station` do pokrycia opłat. [1]",
  "depends_on": ["payout_crypto_initiate_transfer"],
  "outputs": ["backend_awaits_payout_webhook"],
  "related_data": ["circle_payout_id"],
  "visibility": "internal",
  "error_handling": "API Circle zwraca błąd. Wypłata jest oznaczana jako `FAILED`."
},
{
  "id": "path_payout_fiat",
  "title": "Ścieżka wypłaty fiat (na konto bankowe)",
  "type": "group",
  "children": [
    "payout_fiat_check_kyc",
    "condition_kyc_passed",
    "payout_fiat_initiate_kyc_flow",
    "payout_fiat_initiate_transfer",
    "api_call_circle_payout_fiat",
    "backend_awaits_payout_webhook",
    "db_update_withdrawal_record_confirmed",
    "creator_receives_payout_confirmation_email"
  ]
},
{
  "id": "payout_fiat_check_kyc",
  "title": "Sprawdzenie statusu KYC twórcy",
  "type": "backend_logic",
  "description": "Wypłaty fiat mogą wymagać weryfikacji tożsamości. System sprawdza w bazie pole `kyc_status` użytkownika. Jest to kluczowy punkt regulacyjny. [1]",
  "depends_on": ["condition_payout_method"],
  "outputs": ["condition_kyc_passed"],
  "related_data": ["user_id", "kyc_status"],
  "visibility": "internal",
  "error_handling": "Brak."
},
{
  "id": "condition_kyc_passed",
  "title": "Warunek: Czy KYC jest zweryfikowane?",
  "type": "condition",
  "description": "Jeśli status KYC to `VERIFIED`, proces jest kontynuowany. W przeciwnym razie, inicjowany jest przepływ KYC."
}

```

```

    "depends_on": ["payout_fiat_check_kyc"],
    "outputs": ["payout_fiat_initiate_transfer", "payout_fiat_initiate_kyc_flow"],
    "related_data": ["kyc_status"],
    "visibility": "internal",
    "error_handling": "Brak."
  },
  {
    "id": "payout_fiat_initiate_kyc_flow",
    "title": "Inicjacja procesu KYC",
    "type": "group",
    "description": "Złożony pod-proces, w którym użytkownik jest proszony o dostarczenie dokumentów, a system integruje się z usługą weryfikacji tożsamości (potencjalnie od Circle). Wypłata jest wstrzymana do czasu pomyślnej weryfikacji.",
    "children":
  },
  {
    "id": "payout_fiat_initiate_transfer",
    "title": "Backend inicjuje transfer fiat",
    "type": "backend_logic",
    "description": "Backend wywołuje `Circle Payouts API`, przekazując kwotę, walutę i identyfikator beneficjenta (konto bankowe twórcy). [1]",
    "depends_on": ["condition_kyc_passed"],
    "outputs": ["api_call_circle_payout_fiat"],
    "related_data": ["amount", "currency", "beneficiary_id"],
    "visibility": "internal",
    "error_handling": "Brak."
  },
  {
    "id": "api_call_circle_payout_fiat",
    "title": "Wywołanie API Circle Payouts",
    "type": "api_call",
    "description": "Circle wykonuje konwersję USDC na fiat i zleca przelew bankowy. [1]",
    "depends_on": ["payout_fiat_initiate_transfer"],
    "outputs": ["backend_awaits_payout_webhook"],
    "related_data": ["circle_payout_id"],
    "visibility": "internal",
    "error_handling": "API Circle zwraca błąd. Wypłata jest oznaczana jako `FAILED`."
  },
  {
    "id": "backend_awaits_payout_webhook",
    "title": "Backend oczekuje na webhook o wypłacie",
    "type": "system_event",
    "description": "Circle wysyła webhook po pomyślnym zaksięgowaniu transakcji on-chain lub zrealizowaniu przelewu bankowego.",
    "depends_on": ["api_call_circle_payout_onchain", "api_call_circle_payout_fiat"],
    "outputs": ["db_update_withdrawal_record_confirmed"],
    "related_data": ["payout_id", "status_confirmed"],
    "visibility": "internal",
    "error_handling": "Brak webhooka. System może okresowo odpytywać o status wypłaty."
  },
  {
    "id": "db_update_withdrawal_record_confirmed",
    "title": "Aktualizacja rekordu wypłaty na 'Confirmed'",
    "type": "data_store",
    "description": "Po otrzymaniu potwierdzenia, status rekordu w tabeli `Withdrawal` jest zmieniany na `CONFIRMED`.",
    "depends_on": ["backend_awaits_payout_webhook"],
    "outputs": ["creator_receives_payout_confirmation_email"],
    "related_data": ["withdrawal_id"],
    "visibility": "internal",
    "error_handling": "Błąd zapisu do bazy."
  },
  {
    "id": "creator_receives_payout_confirmation_email",
    "title": "Twórca otrzymuje e-mail z potwierdzeniem wypłaty",
    "type": "system_event",
    "description": "Serwis powiadomień wysyła e-mail do twórcy z informacją o zrealizowanej wypłacie.",
    "depends_on": ["db_update_withdrawal_record_confirmed"],
    "outputs": ,

```

```

    "related_data": ["user_email", "amount", "destination_address"],
    "visibility": "internal",
    "error_handling": "Błąd wysyłki e-maila (np. przez SendGrid).",
  },
  {
    "id": "event_tip_confirmed",
    "title": "Zdarzenie: Napiwek potwierdzony",
    "type": "system_event",
    "description": "Wewnętrzne zdarzenie systemowe emitowane po pomyślnym zaksięgowaniu napiwku. Służy jako trigger dla innych, niezależnych modułów.",
    "depends_on": ["db_update_tip_record_confirmed"],
    "outputs": ["notification_service_sends_email", "gamification_service_evaluates_activity", "analytics_service_logs_event"],
    "related_data": ["tip_object"],
    "visibility": "internal",
    "error_handling": "Brak."
  },
  {
    "id": "notification_service_sends_email",
    "title": "Serwis powiadomień wysyła e-mail",
    "type": "backend_logic",
    "description": "Nasłuchuje na zdarzenia (np. `tip.confirmed`, `payout.confirmed`) i wysyła odpowiednie powiadomienia e-mail/push do użytkowników. [1]",
    "depends_on": ["event_tip_confirmed"],
    "outputs":,
    "related_data": ["recipient_email", "template_id", "template_data"],
    "visibility": "internal",
    "error_handling": "Błąd integracji z dostawcą e-maili (np. SendGrid). Błąd jest logowany."
  },
  {
    "id": "gamification_service_evaluates_activity",
    "title": "Serwis gamifikacji ocenia aktywność",
    "type": "backend_logic",
    "description": "Nasłuchuje na zdarzenia (np. `tip.confirmed`) i na ich podstawie ocenia, czy użytkownik (fan lub twórca) kwalifikuje się do zdobycia odznaki, awansu w rankingu lub osiągnięcia innego celu grywalizacyjnego. [1]",
    "depends_on": ["event_tip_confirmed"],
    "outputs":,
    "related_data": ["user_id", "activity_type", "amount"],
    "visibility": "internal",
    "error_handling": "Błąd w logice gamifikacji. Błąd jest logowany, ale nie wpływa na główny przepływ."
  },
  {
    "id": "analytics_service_logs_event",
    "title": "Serwis analityczny loguje zdarzenie",
    "type": "backend_logic",
    "description": "Rejestruje kluczowe zdarzenia systemowe i interakcje użytkowników do celów analitycznych i biznesowych. [1]",
    "depends_on": ["event_tip_confirmed"],
    "outputs":,
    "related_data": ["event_name", "event_properties"],
    "visibility": "internal",
    "error_handling": "Błąd wysyłki do systemu analitycznego. Błąd jest logowany."
  },
  {
    "id": "admin_navigates_to_login_page",
    "title": "Admin przechodzi do strony logowania",
    "type": "user_action",
    "description": "Administrator otwiera dedykowaną stronę logowania do panelu administracyjnego.",
    "depends_on":,
    "outputs": ["admin_submits_credentials"],
    "related_data":,
    "visibility": "admin_only",
    "error_handling": "Brak."
  },
  {
    "id": "admin_submits_credentials",
    "title": "Admin podaje dane logowania",
    "type": "user_action",

```

```

        "description": "Administrator wprowadza login, hasło i ewentualnie kod 2FA.",
        "depends_on": ["admin_navigates_to_login_page"],
        "outputs": ["backend_validates_admin_credentials"],
        "related_data": ["username", "password", "2fa_code"],
        "visibility": "admin_only",
        "error_handling": "Brak."
    },
    {
        "id": "backend_validates_admin_credentials",
        "title": "Backend waliduje dane admina",
        "type": "backend_logic",
        "description": "System sprawdza poprawność danych logowania administratora. Dostęp jest dodatkowo
zabezpieczony, np. przez 2FA i ograniczony do określonych adresów IP. [1]",
        "depends_on": ["admin_submits_credentials"],
        "outputs": ["admin_session_created"],
        "related_data": ["is_valid_credentials"],
        "visibility": "admin_only",
        "error_handling": "Nieprawidłowe dane. Zwracany jest błąd 401. Stosowany jest rate-limiting, aby zapobiec
atakowi brute-force."
    },
    {
        "id": "admin_session_created",
        "title": "Sesja administratora zostaje utworzona",
        "type": "backend_logic",
        "description": "Po pomyślnej weryfikacji, tworzona jest sesja administratora z podwyższonymi
uprawnieniami.",
        "depends_on": ["backend_validates_admin_credentials"],
        "outputs": ["admin_views_dashboard"],
        "related_data": ["admin_jwt_token"],
        "visibility": "admin_only",
        "error_handling": "Brak."
    },
    {
        "id": "admin_views_dashboard",
        "title": "Admin widzi panel administracyjny",
        "type": "user_action",
        "description": "Administrator ma dostęp do panelu z narzędziami do moderacji, konfiguracji i monitoringu
platformy. [1]",
        "depends_on": ["admin_session_created"],
        "outputs": ["admin_configures_platform_fees", "admin_moderates_content"],
        "related_data": ["platform_stats", "moderation_queue"],
        "visibility": "admin_only",
        "error_handling": "Brak."
    },
    {
        "id": "admin_configures_platform_fees",
        "title": "Admin konfiguruje prowizję platformy",
        "type": "user_action",
        "description": "Administrator ustawia globalną stawkę prowizji lub definiuje reguły dla planów premium.
[1]",
        "depends_on": ["admin_views_dashboard"],
        "outputs": ["backend_saves_fee_configuration"],
        "related_data": ["fee_percentage", "premium_plan_rules"],
        "visibility": "admin_only",
        "error_handling": "Brak."
    },
    {
        "id": "backend_saves_fee_configuration",
        "title": "Backend zapisuje konfigurację prowizji",
        "type": "backend_logic",
        "description": "Logika biznesowa zapisuje nowe ustawienia prowizji, które będą stosowane do przyszłych
transakcji.",
        "depends_on": ["admin_configures_platform_fees"],
        "outputs": ["db_update_platform_config"],
        "related_data": ["new_config_object"],
        "visibility": "admin_only",
        "error_handling": "Błąd zapisu konfiguracji."
    },
    {

```

```

      "id": "db_update_platform_config",
      "title": "Aktualizacja konfiguracji w bazie danych",
      "type": "data_store",
      "description": "Nowe ustawienia prowizji są zapisywane w dedykowanej tabeli konfiguracyjnej w bazie danych.",
      "depends_on": ["backend_saves_fee_configuration"],
      "outputs":,
      "related_data":,
      "visibility": "admin_only",
      "error_handling": "Błąd zapisu do bazy."
    },
    {
      "id": "user_session_expires_or_logs_out",
      "title": "Użytkownik wylogowuje się lub sesja wygasa",
      "type": "system_event",
      "description": "Końcowy punkt dla sesji użytkownika. Token JWT jest unieważniany (jeśli stosowana jest lista odwołań) lub po prostu wygasa. Ciasteczko jest usuwane z przeglądarki.",
      "depends_on":,
      "outputs":,
      "related_data": ["user_id"],
      "visibility": "internal",
      "error_handling": "Brak."
    }
  ]
}

```

Schematy Danych i Katalog Komponentów

Aby zapewnić spójność i jednoznaczność implementacji, poniżej przedstawiono definicje kluczowych schematów bazy danych oraz katalog zewnętrznych usług, z którymi integruje się system. Stanowią one pojedyncze źródło prawdy dla deweloperów.

Definicje Schematów Bazy Danych (PostgreSQL)

Schematy te są oparte na analizie dokumentacji i rozszerzone o pola wynikające z mapowania procesów, takie jak

`kyc_status` czy `balance`.

Tabela	Kolumna	Typ Danych (PostgreSQL)	Opis
User	<code>id</code>	<code>UUID</code> (PRIMARY KEY)	Unikalny identyfikator użytkownika.
	<code>email</code>	<code>VARCHAR(255)</code> (UNIQUE)	Adres e-mail użytkownika, unikalny w systemie.
	<code>googleId</code>	<code>VARCHAR(255)</code> (UNIQUE)	Unikalny identyfikator z profilu Google (dla logowania OAuth).
	<code>walletAddressSIWE</code>	<code>VARCHAR(42)</code> (UNIQUE)	Adres portfela Ethereum użyty do logowania przez Sign-In with Ethereum.
	<code>circleWalletId</code>	<code>UUID</code> (UNIQUE)	Unikalny identyfikator portfela w systemie Circle.
	<code>mainWalletAddress</code>	<code>VARCHAR(42)</code>	Adres publiczny portfela USDC na blockchainie (np. Polygon).
	<code>username</code>	<code>VARCHAR(50)</code> (UNIQUE)	Unikalna, publiczna nazwa użytkownika (np. <code>@alias</code>).
	<code>displayName</code>	<code>VARCHAR(100)</code>	Wyświetlana nazwa użytkownika.

	balance	DECIMAL(18, 6)	Lokalne odzwierciedlenie salda USDC. Główne źródło prawdy to Circle, ale cache ułatwia operacje.
	kyc_status	VARCHAR(20)	Status weryfikacji KYC (NOT_STARTED , PENDING , VERIFIED , REJECTED).
	createdAt	TIMESTAMP WITH TIME_ZONE	Data i czas utworzenia konta.
	lastLoginAt	TIMESTAMP WITH TIME_ZONE	Data i czas ostatniego logowania.
Tip	id	UUID (PRIMARY KEY)	Unikalny identyfikator transakcji napiwku.
	fromUserId	UUID (FK to User)	ID fana (może być NULL dla anonimowych napiwków).
	toUserId	UUID (FK to User)	ID twórcy, który otrzymał napiwek.
	amount	DECIMAL(18, 6)	Kwota napiwku w USDC.
	platformFee	DECIMAL(18, 6)	Kwota prowizji pobranej przez platformę.
	netAmount	DECIMAL(18, 6)	Kwota, która trafiła do twórcy (amount - platformFee).
	message	TEXT	Opcjonalna wiadomość od fana.
	txHash	VARCHAR(66)	Hash transakcji on-chain (jeśli dotyczy).
	paymentMethod	VARCHAR(20)	Metoda płatności (USDC_ONCHAIN , USDC_INTERNAL , CARD).
	status	VARCHAR(20)	Status transakcji (INITIATED , PENDING , CONFIRMED , FAILED).
	timestamp	TIMESTAMP WITH TIME_ZONE	Data i czas transakcji.
Withdrawal	id	UUID (PRIMARY KEY)	Unikalny identyfikator wypłaty.
	userId	UUID (FK to User)	ID twórcy zlecającego wypłatę.
	amount	DECIMAL(18, 6)	Kwota wypłaty w USDC.
	destinationAddress	VARCHAR(255)	Adres docelowy (portfel krypto lub dane konta bankowego).
	method	VARCHAR(20)	Metoda wypłaty (CRYPTO_ONCHAIN , FIAT_BANK_TRANSFER).
	status	VARCHAR(20)	Status wypłaty (PENDING , CONFIRMED , FAILED).
	timestamp	TIMESTAMP WITH TIME_ZONE	Data i czas zlecenia wypłaty.
	txHash	VARCHAR(66)	Hash transakcji on-chain (jeśli dotyczy).
	circlePayoutId	UUID	Identyfikator operacji w systemie Circle.

Eksportuj do Arkuszy

Katalog Zewnętrznych Usług i Punktów Integracji

Centralizacja informacji o zależnościach od usług zewnętrznych jest kluczowa dla zarządzania konfiguracją, kluczami API i potencjalnymi punktami awarii.

Usługa	Odpowiedzialny Moduł (Backend)	Kluczowe Używane Endpointy/Funkcje API	Cel Integracji	Dostępność Sandbox
Circle Platform	CircleIntegration (CircleService)	<code>wallets.createWallet</code> , <code>transfers.createTransfer</code> , <code>payments.createPaymentIntent</code> , <code>payouts.createPayout</code>	Podstawowa infrastruktura portfeli, płatności i wypłat. Rdzeń systemu.	Tak, pełne środowisko sandbox.
Google	Auth (GoogleStrategy)	Google OAuth 2.0 API	Uwierzytelnianie i rejestracja twórców.	Tak, poprzez projekt w Google Cloud Console.
Twitch / YouTube	Auth	OAuth 2.0 API	(Planowane) Alternatywne metody uwierzytelniania dla twórców.	Tak.
SendGrid (lub podobne)	Notifications	Email API	Wysyłanie transakcyjnych wiadomości e-mail (potwierdzenia, powiadomienia).	Tak, darmowy plan z limitami.
Sentry	(Integracja globalna)	Sentry SDK dla NestJS i React	Agregacja i monitoring błędów aplikacji w czasie rzeczywistym.	Tak, darmowy plan dla deweloperów.
Dostawca Blockchain Node (np. Alchemy)	Tips/Payments	JSON-RPC API, WebSockets (dla subskrypcji zdarzeń)	Monitorowanie transakcji on-chain (alternatywa dla webhooków Circle).	Tak, darmowy dostęp do sieci testowych (np. Polygon Mumbai).

Wnioski i Rekomendacje Architektoniczne

Analiza i mapowanie systemu TipJar prowadzą do kilku kluczowych wniosków, które powinny kierować procesem implementacji:

1. **Circle API jako Rdzeń Systemu:** Moduł **CircleService** jest sercem całej platformy. Odpowiada za zarządzanie portfelami, transfery i płatności, co czyni go najbardziej krytyczną zależnością. Jego rozwój i gruntowne przetestowanie w środowisku sandbox Circle powinno być absolutnym priorytetem. Wszystkie inne funkcje, od napiwków po wypłaty, opierają się na poprawnej implementacji tej warstwy.

2. **Konieczność Przyjęcia Modelu Asynchronicznego:** Operacje takie jak tworzenie portfela dla nowego użytkownika czy wysyłanie powiadomień nie mogą blokować głównego wątku aplikacji. Jak wskazano w dokumentacji, użycie kolejki zadań (np. BullMQ z Redisem) jest nie tylko zaleceniem, ale koniecznością dla zapewnienia responsywności i dobrego doświadczenia użytkownika. Umożliwia to natychmiastowe zalogowanie twórcy, podczas gdy jego portfel jest tworzony w tle.
3. **Zarządzanie Stanem dla Warunkowego UI:** Frontend musi być przygotowany na obsługę stanów przejściowych. Użytkownik może być zalogowany, ale jego portfel może być wciąż w trakcie tworzenia. Wypłata może być wstrzymana w oczekiwaniu na weryfikację KYC. Te scenariusze wymagają solidnej architektury zarządzania stanem po stronie klienta (np. z użyciem React Context, Zustand lub Redux Toolkit), która będzie potrafiła odpytywać backend o status i dynamicznie aktualizować interfejs.
4. **Ujednolicona Strategia Logowania i Monitoringu:** Ze względu na rozproszoną naturę systemu (frontend, backend, kolejka zadań, liczne API zewnętrzne), wdrożenie scentralizowanej strategii logowania (np. ELK Stack, Datadog) oraz śledzenia błędów (Sentry) jest fundamentalne dla utrzymania i debugowania platformy. Bez tego, diagnozowanie problemów w przepływach obejmujących wiele komponentów będzie niezwykle trudne.
5. **Rekomendacja Wdrożenia Etapowego:** Aby zminimalizować ryzyko, zaleca się wdrożenie platformy w kilku fazach:
 - **Faza 1 (MVP):** Skupienie się na podstawowym cyklu Web3. Obejmuje rejestrację twórców, przekazywanie napiwków wyłącznie metodą krypto (on-chain i wewnętrzne) oraz wypłaty krypto. Ta faza waliduje kluczową pętlę wartości platformy.
 - **Faza 2:** Wprowadzenie płatności kartą (fiat on-ramp). Ta faza dodaje znaczącą złożoność związaną z integracją bramek płatniczych i modelem biznesowym (pokrywanie opłat).
 - **Faza 3:** Wprowadzenie wypłat fiat na konta bankowe (fiat off-ramp) wraz z wymaganymi procesami KYC/AML. Jest to najbardziej złożony etap pod względem regulacyjnym i powinien być wdrażany po ustabilizowaniu poprzednich funkcji.

Postępowanie zgodnie z tym modelem logicznym i powyższymi rekomendacjami zapewni solidne fundamenty dla budowy skalowalnej, bezpiecznej i przyjaznej dla użytkownika platformy mikropłatności TipJar.